IBM Quantum

Dynamic Circuits

Fatima Bint-e-Munir

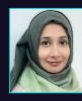
QISKIT 2026 2026 FALL FEST

Speaker Introduction

ABOUT:

Fatima Bint-e-Munir

Assistant Professor,
Department of Physics,
Government College University, Lahore



Interested in everything Physics
Nonlinear Phenomenon in Fusion devices, Space Plasmas and NLTLs
Quantum Hamiltonian Simulations

QISKIT FALL FEST

Dynamic Circuits

measurement, feedforward, and control flow inside a quantum circuit

CR MCM IF DC

Built with content from the IBM Quantum Learning platform & Qiskit documentation

Fall Fest Roadmap

You Are Here: Talk 8 of the Series

✓ Intro to QC + Qiskit setup & Vectors and Matrices	✓ Single-qubit gates + superposition	✓ Measurement, Circuit & Qiskit Patterns	✓ Multi-qubit systems + entanglement
✓ Circuit optimization & transpilation I	✓ Noise Description in Quantum Circuits	✓ Real Hardware	8 Dynamic circuits — WE ARE HERE

60-Second Recap

What You Already Know

From earlier talks — ideas we'll lean on heavily today.

Measurement gives a classical bit
qc.measure(qubit, clbit) writes a 0 or 1 into a classical bit — you've used used this throughout the the series.

The Born rule
$P = |\text{amplitude}|^2$
Measurement is random, but the odds are exact.

CNOT and reset
CNOT builds entanglement (Talk 4).
Reset returns a qubit to $|0\rangle$ — we'll use it a lot today.

GHZ states
$(|0\ldots0\rangle + |1\ldots1\rangle)/\sqrt{2}$
Built with H and a cascade of CNOTs in Talk 4.

Today, all four of these ideas come together at once — that's what makes a circuit "dynamic."

Today's Session

Where We're Headed

01	Classical registers	Where measurement outcomes actually live
02	Mid-circuit measurement	Measuring before the circuit ends — without ending everything else
03	Conditional logic — if_test	Turning a measurement outcome into a decision
04	Building a dynamic circuit	A GHZ state built with measurement and feedforward

▶ By The End Of This Talk

Learning Objectives

1. Explain what a classical register is, and why measurement outcomes need somewhere to live
2. Describe mid-circuit measurement, and why measuring one qubit doesn't collapse the whole computation
3. Write a conditional operation with if_test, and explain what classical feedforward means
4. Define what makes a circuit genuinely "dynamic" — and build one, step by step: a GHZ state using state using measurement, reset, and feedforward

▶ Dynamic Circuits

What Is a Dynamic Circuit?

A measurement outcome can determine a later operation in the same circuit.

STATIC CIRCUIT	DYNAMIC CIRCUIT
A fixed sequence	**Measure, decide, continue**
Prepare → apply gates → measure Measurement produces the final readout. The gate sequence is fixed in advance.	Prepare → measure Choose a gate → continue Example: apply X if the outcome is 1. Skip X if it is 0. The choice happens during execution.

Classical feedforward uses a stored outcome to choose a later quantum operation.

▶ Dynamic Circuits

Dynamic Circuits: Uses & Requirements

Measurement and feedback let a circuit adapt as it runs, somewhat like GPS navigation.

WHY WE USE THEM	WHAT THEY REQUIRE
Teleportation: select corrections from measurement outcomes.	Measurement during the circuit
Reset and reuse: prepare a qubit for another role.	Classical bits to store outcomes
Error correction: choose corrections from syndrome outcomes.	Conditional control of later gates
	Backend support and fast feedback

Where do we store measurement outcomes? In classical bits and registers.

▶ PART 1

Classical Registers

Every measurement outcome has to live somewhere — this is that somewhere

CR

▶ The Basics

What Is a Classical Register?

A quantum circuit needs somewhere to put the classical 0s and 1s that come out of measurement — that's a classical that's a classical register.

QUANTUM REGISTER	CLASSICAL REGISTER
`QuantumRegister(n)`	`ClassicalRegister(n)`
A collection of n qubits. Each one can hold a superposition, and can be entangled with others.	A collection of n ordinary bits (cbits). Each one holds a plain 0 or 1 — no superposition, no entanglement.

Once a qubit is measured, its outcome is written into a cbit — this is how a circuit keeps its quantum and classical parts separate. separate.

▶ Why It Matters

Why Bother Grouping Bits?

You could track individual cbits one at a time — but a register gives you a name, an order, and a way to address them together.

A name and a size	ClassicalRegister(4, "c") reads naturally — "c" is a 4-bit register, not four anonymous bits.
An order	c[0], c[1], c[2]… — the same little-endian ordering you already know from multi-qubit kets.
Something to test	Conditions like if_test((c, 1)) check a whole register's value at once, not bit by bit.

▶ In Qiskit

Creating & Using Registers

Qiskit

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

qr = QuantumRegister(2, "q")
cr = ClassicalRegister(2, "c")
qc = QuantumCircuit(qr, cr)

qc.measure(qr[0], cr[0])   # write qubit 0's outcome into c[0]
```

You've already been doing this

QuantumCircuit(2, 2) is shorthand — it silently creates a 2-qubit QuantumRegister and a 2-bit ClassicalRegister for you. Today we're just making that explicit.

▶ Predict Before You Peek

Try It — Reading a Register

```
cr = ClassicalRegister(3, "c")    →    qc.measure(qr[1], cr[1])
```

If qubit 1 is measured as |1⟩, and every other bit in c starts at 0, what does the register cr read as — as a read as — as a bitstring, and as an integer?

Remember little-endian: c[0] is the rightmost digit. c reads 010 — which is the integer 2.

▶ PART 2

Mid-Circuit Measurement

The next leap in Quantum AI isn't "more qubits" — it's qubits that can listen, respond, and adapt in real and adopt in real time.

MCM

▶ The Shift

From Terminal to Mid-Circuit

Every measurement you've done so far has been the very last step. Dynamic circuits move it into the middle.

TERMINAL MEASUREMENT (familiar)

`prep → gates → measure`

The circuit is finished the moment you measure. Nothing Nothing quantum happens afterward.

MID-CIRCUIT MEASUREMENT (new)

`prep → gates → measure → more gates`

The measured qubit gives up its result — but the circuit keeps going, on that qubit or others.

The natural question: if I measure one qubit in the middle, what happens to the qubits I didn't measure?

▶ The Key Nuance

Measuring One Qubit, Not All of Them

"Measurement doesn't collapses the whole quantum circuit"— only the measured qubit is projected.

`|+⟩ ⊗ |0⟩  →  measure the |+⟩ qubit  →  (|0⟩ or |1⟩) ⊗ |0⟩    (still evolving)`

Here the two qubits start as a product state, so only the measured qubit collapses to a definite classical outcome. The other qubit keeps its state, and gates can still act on it. (Entangled qubits behave differently: collapsing one updates its partner — we'll see that in the GHZ build.)

Why this matters for dynamic circuits

This is exactly what dynamic circuits exploit: measure a qubit early, get a classical answer, and keep computing with everything else — including using that classical answer to steer what comes next.

▶ In Qiskit

Mid-Circuit Measurement in Code

The same qc.measure() call as before — just placed before the circuit ends.

Qiskit

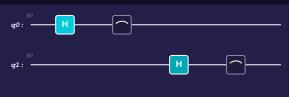
```
qr = QuantumRegister(2, "q")
cr = ClassicalRegister(2, "c")
qc = QuantumCircuit(qr, cr)

qc.h(0)
qc.measure(0, 0)          # mid-circuit: q0 is measured, q1 keeps going
qc.h(1)                   # q1 is untouched by the measurement above
qc.measure(1, 1)
```

The order of operations in your code is the order the hardware runs them in — that ordering now genuinely matters.

▶ Seeing It

What This Looks Like



Right now, these two measurements don't talk to each other — q_1 doesn't know or care what q_0 measured.
That's exactly what's going to change.

▶ Predict Before You Peek

Try It — Measure, Then Keep Going

```
|0⟩⊗|0⟩ → H on q0 → measure q0 → H on q1 → measure q1
```

Does measuring q_0 change the probabilities you'll get when you later measure q_1?

No. q_0 and q_1 were never entangled — when q_0 is measured, the pair is the product state $|+\rangle \otimes |0\rangle$. Measuring q_0 only affects q_0; q_1's own H and measurement give 50/50 regardless.

▶ PART 3

Conditional Logic — if_test

Turning a measurement outcome into an actual decision the circuit makes

IF

▶ The Concept

Classical Feedforward

Once a measurement outcome exists as a classical bit, the circuit can branch on it — this is called feedforward

```
quantum state → measure → classical bit c → if c = 1, apply X;
otherwise, apply I
```

A hybrid computation

This makes a dynamic circuit genuinely hybrid: quantum evolution, a measurement, classical processing, and a follow-up quantum operation, all within one execution — not four separate programs.

▶ The Syntax

Meet if_test

Qiskit's structured way to write a conditional quantum operation.

Qiskit

```
with qc.if_test((cr, 1)):
    qc.x(1)
```

Read literally: "if *cr* equals 1, do this"

if_test returns a context manager — you use it in a 'with' block, exactly like the example above.

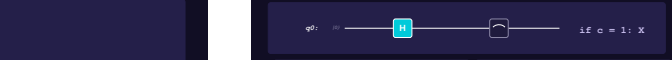
above.

The X gate inside only executes on shots where the register *cr* measured 1.

▶ The Payoff

Turning Randomness Into Certainty

A single if_test can undo the randomness measurement introduces — this is the "aha" moment of dynamic circuits.



Branch c = 0

```
|0⟩ → H → |+⟩ → measure → |0⟩
no correction needed → |0⟩
```

Branch c = 1

```
|0⟩ → H → |+⟩ → measure → |1⟩
apply X (if_test fires) → |0⟩
```

The measurement was random — but the FINAL state is |0⟩ every single time.

In Qiskit

Writing It Out

Qiskit

```
qc = QuantumCircuit(1, 1)
qc.h(0)
qc.measure(0, 0)

with qc.if_test((qc.clbits[0], 1)):
    qc.x(0)

qc.measure(0, 0)   # confirm: this always reads 0
```

This exact pattern — measure, then if_test a correction — is the single most common building block in dynamic circuits. dynamic circuits.

Predict Before You Peek

Try It — Fixing a Different Outcome

`|1⟩ → H → measure → c  —  goal: always end in |1⟩`

What if_test condition and gate would guarantee the qubit ends in |1⟩, no matter what c measures? measures?

If c == 0: apply X. H|1⟩ = |−⟩, which still measures 0 or 1 with 50/50 odds — so flip only on the 0 branch, leaving 1 alone. leaving 1 alone.

PART 4

Dynamic Circuits, Step by Step

Everything from Parts 1–3, working together — built one gate at a time

DC

The Formal Picture

What Actually Makes a Circuit "Dynamic"?

Not just "it has a classical register and mid-circuit measurements" — something more specific.

A dynamic circuit is one where a runtime measurement result is available early enough to influence subsequent operations — within the same execution.

The diagnostic question

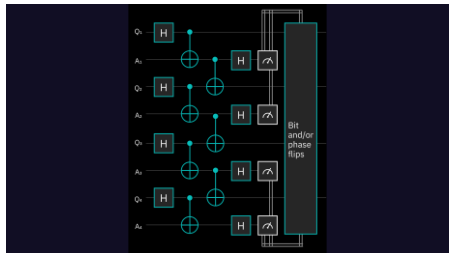
"Does a measurement result obtained during the circuit affect what happens later in the same execution?" If yes, it's dynamic. Parts 1–3 gave you every ingredient — now we assemble them.

Compare & Contrast

Static vs. Dynamic Circuits

Feature	Static circuit	Dynamic circuit
Execution path	Fixed in advance	Can depend on runtime results
Measurement	Usually terminal	Can occur mid-circuit
Feedback	None	Classical feedforward to later gates
Reset / reuse	Rarely used	Lets a measured qubit be reused

A useful test: "same fixed script every time" vs "can genuinely branch."



Today's Build

The Goal: A GHZ State in Constant Depth

In Talk 4, the GHZ circuit's depth grew with every added qubit. With measurement and feedforward, it doesn't have to.

`measure → feed forward → reset → reuse`

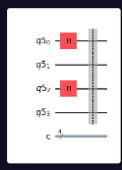
The idea

Measure the in-between qubits mid-circuit, then use their classical results to correct the rest — so the CNOT layers never have to spread across the register one qubit at a time. The measured helpers are reset and reused to finish the state.

We'll build a 4-qubit version, following IBM's "Efficient Long-Range Entanglement Using Dynamic Circuits" (Bäumer et al., PRX Quantum, 2024). At 4 qubits it isn't shorter — measurements take time too; the payoff comes as n grows.

Step 1 of 6

Superpose the Data Qubits



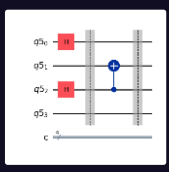
H on q_0 and q_2

Four qubits total: two "data" qubits (q_0, q_2) that will end up will end up holding the GHZ state, plus two helper qubits qubits (q_1, q_3) in between.

H turns q_0 and q_2 into $|+\rangle$ — two independent "coins," each "coins," each equally likely to be 0 or 1.

Step 2 of 6

Build the First Bridge



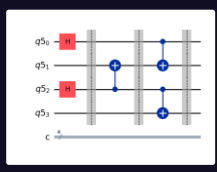
CX(q_2, q_3)

One CNOT, control q_2, target q_3. This is the exact Bell-exact Bell-pair recipe from Talk 4 — q_1 and q_2 are now are now entangled.

q_3 is the "bridge" — it will carry information between the two halves of the circuit.

Step 3 of 6

Fold In the Data Qubits



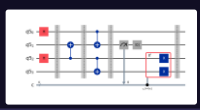
CX(q_0,q_1), CX(q_2,q_3)

Two more CNOTs fold q_0 into the bridge and bridge and entangle q_2 onto the edge qubit qubit q_0.

The bridge q_3 now holds the PARITY of the two coins — it secretly knows whether q_0 and q_2 agree.

Step 4 of 6 — The Payoff

Measure the Bridge, Then Correct

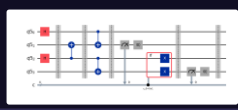


This is dynamic circuits, in one gate

Measuring q_3 gives a random bit m_1 — it tells us whether the two coins agreed. Reset q_3, then if_test: only if $m_1 = 1$, flip q_1 and $m_1 = 1$, flip q_2 and q_3. Both branches now agree — the randomness has been corrected away, leaving a deterministic 3-qubit deterministic 3-qubit GHZ on q_0, q_1, q_2.

Step 5 of 6

Measure and Reset the Edge Qubit

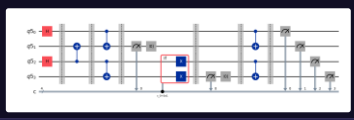


Pinning down the shared value

q_3 is measured (would the same classical bit) and reset. Its outcome m_2 fixes the shared value: $q_0 = q_2 = m_2$ on that shot. on that shot. (This measurement collapses the GHZ superposition on q_0, q_2, q_3 — more on that after the results.) results.)

▶ Step 6 of 6

The Final Layer — and the Full Circuit

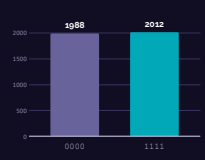


$CX(q_0,q_1)$ and $CX(q_2,q_3)$ bring the two reset helper qubits back in sync with the data — all four qubits now read the same value, read the same value, giving 0000 or 1111 on every shot.

▶ Run It

The Result: Only 0000 and 1111

4,000 shots of the circuit we just built, sampled on an ideal simulator.



What the histogram shows

Out of 16 possible 4-bit outcomes, only 0000 and 1111 ever appear — the Z-basis signature of a GHZ state.

One caveat

Z-basis counts can't tell a GHZ superposition from a 50/50 mix of 0000 and 1111. The real test is an X-basis parity check, (XXXX) = +1. Step 5 measures q_0, collapsing the state, so this build scores 0; dropping Step 5 and the final $CX(q_0,q_1)$ scores +1.

▶ Typical Use Cases

Why Dynamic Circuits Matter

Quantum error correction	Measure syndromes, decode, correct, repeat — QEC is inherently a dynamic-circuit protocol.
Quantum teleportation	A Bell measurement plus classical feedforward moves a state without ever measuring it directly.
Long-range entanglement	Constant-depth GHZ states and long-range CNOTs on nearest-neighbour hardware — today's build in miniature.
Qubit reuse & adaptive algorithms	Reset and reuse qubits, or retry and change course based on what you observe observe mid-run.

▶ Recap

Let's Check the List

- ✓ Explain what a classical register is, and why measurement outcomes need somewhere to live
- ✓ Describe mid-circuit measurement, and why measuring one qubit doesn't collapse the whole computation
- ✓ Write a conditional operation with if_test, and explain what classical feedforward means
- ✓ Define what makes a circuit genuinely "dynamic" — and build one, step by step: a GHZ state using state using measurement, reset, and feedforward

▶ Keep Exploring

Resources

Classical feedforward and control flow
IBM's guide to if_test, switch, for_loop, and while_loop
quantum.cloud.ibm.com/docs/en/guides/classical-feedforward-and-control-flow

Execute dynamic circuits
Hardware considerations, limitations, and backend selection
quantum.cloud.ibm.com/docs/en/guides/execute-dynamic-circuits

Long-range entanglement with dynamic circuits
IBM's tutorial: a long-range CNOT built from Bell pairs, measurement, and feedforward
quantum.cloud.ibm.com/docs/en/tutorials/long-range-entanglement

Efficient Long-Range Entanglement Using Dynamic Circuits
Baumer et al., PRX Quantum 5, 030339 (2024) — the GHZ construction behind today's build
journals.aps.org/prxquantum/abstract/10.1103/PRXQuantum.5.030339

▶ THANK YOU

Questions?

No pressure, no wrong questions — if measure-then-correct still feels like magic, that means you're paying you're paying attention.

CR MCM IF DC